

LAPORAN
WORKSHOP SISTEM INFORMASI WEB SERVER
(TIF330805)
MINGGU 8 PERTEMUAN 1
SEMESTER III



BKPM ACARA 15

Nama:
SEPTIAN DWI RAMADHAN PUTRA

NIM:
E41250282

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)
TEKNIK INFORMATIKA KAB. NGANJUK
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI JEMBER
TAHUN 2026

PENDAHULUAN

1. Latar Belakang

Pada praktikum sebelumnya, data mahasiswa hanya dapat diakses melalui halaman web yang dirender oleh server. Dalam pengembangan aplikasi modern, data sering dibutuhkan oleh aplikasi lain, misalnya aplikasi mobile atau frontend terpisah. Aplikasi tersebut tidak mengakses database secara langsung, melainkan meminta data melalui API (Application Programming Interface).

Pada acara ini dibuat API Data Mahasiswa sederhana yang mengambil data dari MySQL dan mengirimkannya dalam format JSON melalui HTTP. API dibangun di atas lapisan yang sudah dibuat pada acara sebelumnya (Service dan Repository) sehingga validasi, prepared statement, exception handling, dan logging tetap dipakai ulang. Pengujian dilakukan melalui browser dan Postman.

2. Tujuan

1. Menjelaskan konsep API dan JSON serta hubungan keduanya.
2. Membuat endpoint `api/mahasiswa.php` yang mengambil data mahasiswa dari MySQL menggunakan PDO.
3. Mengubah hasil query menjadi response JSON dengan struktur success, message, dan data.
4. Menambahkan endpoint GET daftar, GET berdasarkan id, dan POST dengan body JSON.
5. Menguji API menggunakan browser dan Postman.

3. Dasar Teori Singkat

API. Mekanisme yang memungkinkan satu aplikasi berkomunikasi dengan aplikasi atau sistem lain. Pada aplikasi web, komunikasi API umumnya memakai protokol HTTP: client mengirim request, API memprosesnya (misalnya mengambil data dari database), lalu mengirim response.

HTTP Method. GET untuk mengambil data, POST untuk menambah data, PUT/PATCH untuk mengubah data, dan DELETE untuk menghapus data. Pada acara ini digunakan GET dan POST.

JSON. JavaScript Object Notation, format pertukaran data yang ringan dan mudah dibaca. Data berbentuk pasangan key dan value. Object ditulis dengan { }, sedangkan kumpulan data ditulis dengan array [].

JSON pada PHP. `json_encode()` mengubah array PHP menjadi JSON, sedangkan `json_decode()` mengubah JSON menjadi array PHP. Header Content-Type: application/json memberi tahu client bahwa response berupa JSON.

Hubungan API dan JSON. API adalah jalan komunikasi antar aplikasi, sedangkan JSON adalah salah satu format data yang digunakan dalam komunikasi tersebut.

4. Alat dan Bahan

- Laptop/PC
- Web Browser
- Visual Studio Code
- XAMPP/Laragon (Apache + MySQL/MariaDB)
- PHP 8.2
- Postman
- phpMyAdmin dan file `acara_7.sql`
- Git
- Modul Praktikum (BKPM Acara 15)

HASIL PRAKTIK

1. Persiapan Lapisan Data

API memakai ulang Service dan Repository agar aturan bisnis tidak ditulis dua kali. Tiga tambahan kecil diperlukan: method toArray() pada Entity Mahasiswa untuk mengubah object menjadi array yang siap di-json_encode (atribut Entity tetap private), method findWithProdi() pada Repository untuk mengambil satu data lengkap dengan nama prodi (JOIN dan prepared statement), dan method detail() pada Service sebagai pintu masuknya.

```
// Representasi array untuk response JSON API (BKPM Acara 15). Sengaja
// dibuat di Entity agar Controller/API tidak perlu mengakses atribut private.
public function toArray(): array
{
    return [
        'id'          => $this->id,
        'nim'         => $this->nim,
        'nama'        => $this->nama,
        'prodi_id'    => $this->prodiId,
        'prodi'       => $this->prodiNama,
        'angkatan'    => $this->angkatan,
        'status'      => $this->status,
    ];
}

// ===== SETTER + VALIDASI =====
```

Gambar 3. Entity Mahasiswa: method toArray()

```
// Satu mahasiswa berdasarkan id, lengkap dengan nama prodi (JOIN).
public function findWithProdi(int $id): ?Mahasiswa
{
    $stmt = $this->db->prepare(
        "SELECT m.*, p.nama AS prodi_nama
        FROM mahasiswa m
        JOIN prodi p ON m.prodi_id = p.id
        WHERE m.id = :id"
    );
    $stmt->execute(['id' => $id]);
    $row = $stmt->fetch();

    return $row ? $this->hydrate($row) : null;
}
```

Gambar 4. MahasiswaRepository: method findWithProdi()

```
// Satu mahasiswa lengkap dengan nama prodi (dipakai oleh API).
public function detail(int $id): ?Mahasiswa
{
    return $this->repo->findWithProdi($id);
}
```

Gambar 5. MahasiswaService: method detail()

2. Endpoint api/mahasiswa.php

Endpoint dibuat pada folder api/ di akar project, sehingga dapat diakses melalui <http://localhost/si-akademik/api/mahasiswa.php> sesuai modul. File api/.htaccess

menambahkan alamat tanpa ekstensi (/api/mahasiswa) sesuai penulisan pada tugas pengembangan, dengan parameter query tetap diteruskan.

```
# Mengizinkan alamat tanpa ekstensi: /api/mahasiswa -> /api/mahasiswa.php
# Parameter query (?id=1) tetap dibawa (QSA).
RewriteEngine On
RewriteRule ^mahasiswa/?$ mahasiswa.php [QSA,L]
```

Gambar 6. api/.htaccess untuk alamat /api/mahasiswa

Fungsi `respond()` menjadi satu-satunya jalan keluar response. Fungsi ini mengatur HTTP status code, menyusun struktur success, message, data (dan errors bila ada), lalu mengubahnya menjadi JSON dengan `json_encode()`. Header Content-Type: application/json dipasang di awal file.

```
function respond(int $status, bool $success, string $message, $data = null, ?array
$errors = null): void
{
    http_response_code($status);

    $body = [
        'success' => $success,
        'message' => $message,
        'data'     => $data,
    ];
    if ($errors !== null) {
        $body['errors'] = $errors;
    }

    echo json_encode(
        $body,
        JSON_UNESCAPED_UNICODE | JSON_UNESCAPED_SLASHES | JSON_PRETTY_PRINT |
        JSON_INVALID_UTF8_SUBSTITUTE
    );
    exit;
}

function buildService(): MahasiswaService
{
    $db = Database::getInstance();

    return new MahasiswaService(new MahasiswaRepository($db), new
    ProdiRepository($db));
}
```

Gambar 7. Fungsi `respond()` dan `buildService()`

Warning PHP dicatat ke `app.log`, dan exception yang tidak tertangkap (misalnya MySQL mati) menghasilkan response 500 berisi pesan umum, sehingga detail error database tidak pernah dikirim ke client.

```
set_error_handler(function (int $severity, string $message, string $file, int $line):
bool {
    if (!(error_reporting() & $severity)) {
        return false;
    }
}
```

```

        Logger::warning('API PHP error: ' . $message . ' (' . basename($file) . ':' .
        $line . ')');
        return true;
    });

    set_exception_handler(function (Throwable $e): void {
        Logger::error('API: exception tidak tertangani', $e);
        respond(500, false, 'Terjadi kesalahan pada server.');
```

Gambar 8. Handler error pada API

Endpoint GET tanpa parameter mengembalikan daftar mahasiswa. Dengan parameter id, endpoint mengembalikan satu data atau 404 bila tidak ditemukan, dan 400 bila id bukan angka bulat positif. Parameter q (pencarian nama atau NIM) ditambahkan karena fungsi pencarian sudah tersedia pada Repository.

```

function handleGet(): void
{
    $service = buildService();

    if (isset($_GET['id'])) {
        $id = $_GET['id'];

        if (!is_string($id) || !ctype_digit($id) || (int) $id <= 0) {
            respond(400, false, 'Parameter id harus berupa angka bulat positif.');
```

Gambar 9. handleGet(): daftar dan satu data

Endpoint POST membaca body mentah dari php://input lalu mengubahnya dengan json_decode(). Body harus berupa JSON object yang valid, dan setiap nilai harus berupa teks atau angka. Data kemudian diserahkan ke MahasiswaService::create(), sehingga aturan validasi sama dengan form web. Hasilnya dipetakan ke status code: 201 bila berhasil (dengan header Location), 409 bila NIM sudah terdaftar, 400 bila data tidak valid, dan 500 bila terjadi kesalahan server.

```

function handlePost(): void
{
    $raw    = file_get_contents('php://input');
    $input  = json_decode($raw === false ? '' : $raw, true);

    // Body harus berupa JSON object, bukan teks biasa, angka, atau array list.
    if (json_last_error() !== JSON_ERROR_NONE || !is_array($input)
        || ($input !== [] && array_keys($input) !== range(0, count($input) - 1))) {
        respond(400, false, 'Body harus berupa JSON object yang valid.');
```

```

    }

    // Nilai yang dikirim harus skalar (teks/angka), bukan array/object bersarang.
    $errors = [];
    foreach (['nim', 'nama', 'prodi_id', 'angkatan', 'status'] as $key) {
        if (isset($input[$key]) && !is_scalar($input[$key])) {
            $errors[$key] = 'Nilai harus berupa teks atau angka.';
        }
    }
    if (!empty($errors)) {
        respond(400, false, 'Request tidak valid.', null, $errors);
    }

    $service = buildService();
    $result  = $service->create($input);

    if ($result['success']) {
        $mhs = $service->detail((int) $result['id']);

        $path = parse_url($_SERVER['REQUEST_URI'] ?? '', PHP_URL_PATH) ?:
'/api/mahasiswa.php';
        header('Location: ' . $path . '?id=' . (int) $result['id']);

        respond(201, true, 'Data mahasiswa berhasil ditambahkan', $mhs ? $mhs-
>toArray() : null);
    }

    $errors = $result['errors'];

    // Kesalahan server (database): Service sudah mencatatnya ke app.log.
    if (isset($errors['general'])) {
        respond(500, false, $errors['general']);
    }

    // NIM duplikat = konflik dengan data yang sudah ada.
    if (count($errors) === 1 && ($errors['nim'] ?? '') === 'NIM sudah terdaftar.') {
        respond(409, false, 'NIM sudah terdaftar.', null, $errors);
    }

    respond(400, false, 'Data tidak valid.', null, $errors);
}

```

Gambar 10. handlePost(): menerima JSON dan membuat data

Method selain GET dan POST ditolak dengan status 405 dan header Allow. PUT dan DELETE akan dikembangkan pada acara berikutnya (RESTful API).

```

$method = $_SERVER['REQUEST_METHOD'] ?? 'GET';

if ($method === 'GET') {
    handleGet();
} elseif ($method === 'POST') {

```

```

    handlePost();
} else {
    header('Allow: GET, POST');
    respond(405, false, 'Method tidak diizinkan. Gunakan GET atau POST.');
```

Gambar 11. Pemilihan handler berdasarkan HTTP method

3. Daftar Endpoint dan Status Code

Method	URL	Fungsi	Status
GET	/api/mahasiswa.php	Daftar mahasiswa	200; 500 jika server error
GET	/api/mahasiswa.php?id=1	Satu mahasiswa	200; 404 tidak ada; 400 id tidak valid
GET	/api/mahasiswa.php?q=budi	Cari nama atau NIM	200
POST	/api/mahasiswa.php	Tambah mahasiswa (body JSON)	201; 400 tidak valid; 409 NIM sudah ada; 500
Lainnya	/api/mahasiswa.php	Tidak didukung	405

Tabel 3. Endpoint API Mahasiswa

4. Contoh JSON Response

Contoh berikut menunjukkan struktur response berdasarkan data awal pada acara_7.sql. Untuk laporan akhir, bandingkan dengan response yang tampil pada Postman.

GET /api/mahasiswa.php (dipersingkat menjadi dua data):

```

{
  "success": true,
  "message": "Data berhasil diambil",
  "data": [
    {
      "id": 1,
      "nim": "2401001",
      "nama": "Budi Santoso",
      "prodi_id": 1,
      "prodi": "Teknik Informatika",
      "angkatan": 2024,
      "status": "aktif"
    },
    {
      "id": 2,
      "nim": "2401002",
      "nama": "Ani Wijaya",
      "prodi_id": 3,
      "prodi": "Teknik Komputer",
      "angkatan": 2025,
      "status": "aktif"
    }
  ]
}
```

Gambar 12. Contoh response GET daftar mahasiswa (status 200)

GET /api/mahasiswa.php?id=1:

```
{
  "success": true,
  "message": "Data berhasil diambil",
  "data": {
    "id": 1,
    "nim": "2401001",
    "nama": "Budi Santoso",
    "prodi_id": 1,
    "prodi": "Teknik Informatika",
    "angkatan": 2024,
    "status": "aktif"
  }
}
```

Gambar 13. Contoh response GET satu mahasiswa (status 200)

POST /api/mahasiswa.php dengan body JSON:

```
{
  "nim": "2401004",
  "nama": "Dewi",
  "prodi_id": 1,
  "angkatan": 2026,
  "status": "aktif"
}
```

Gambar 14. Contoh body request POST

```
{
  "success": true,
  "message": "Data mahasiswa berhasil ditambahkan",
  "data": {
    "id": 5,
    "nim": "2401004",
    "nama": "Dewi",
    "prodi_id": 1,
    "prodi": "Teknik Informatika",
    "angkatan": 2026,
    "status": "aktif"
  }
}
```

Gambar 15. Contoh response POST berhasil (status 201 Created)

Contoh response gagal:

```
{
  "success": false,
  "message": "Data mahasiswa tidak ditemukan.",
  "data": null
}
```

Gambar 16. GET id yang tidak ada (status 404)

```
{
  "success": false,
  "message": "NIM sudah terdaftar.",
  "data": null,
  "errors": {
    "nim": "NIM sudah terdaftar."
  }
}
```

Gambar 17. POST dengan NIM yang sudah terdaftar (status 409)

```
{
  "success": false,
  "message": "Data tidak valid.",
  "data": null,
  "errors": {
    "nim": "NIM harus berupa angka.",
    "nama": "Nama wajib diisi.",
    "prodi_id": "Program studi tidak ditemukan.",
    "angkatan": "Angkatan harus berupa tahun antara 2000 dan 2100.",
    "status": "Status tidak valid."
  }
}
```

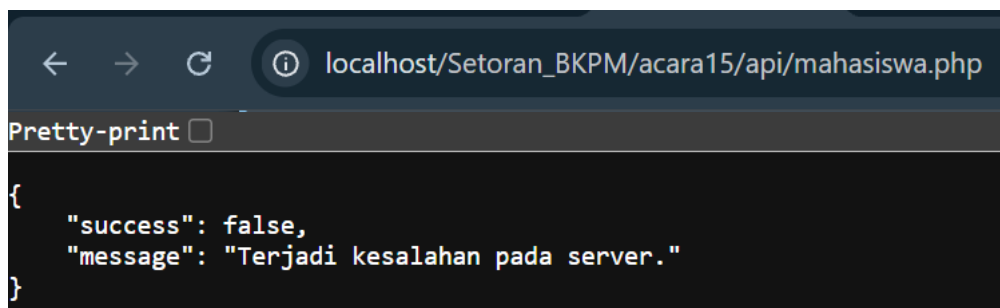
Gambar 18. POST dengan data tidak valid (status 400)

5. Pengujian

Pengujian dilakukan pertama melalui browser untuk request GET, kemudian menggunakan Postman. Koleksi pengujian (docs/postman/si-akademik-api.postman_collection.json) berisi 11 request dengan pengecekan status code otomatis pada tab Tests.

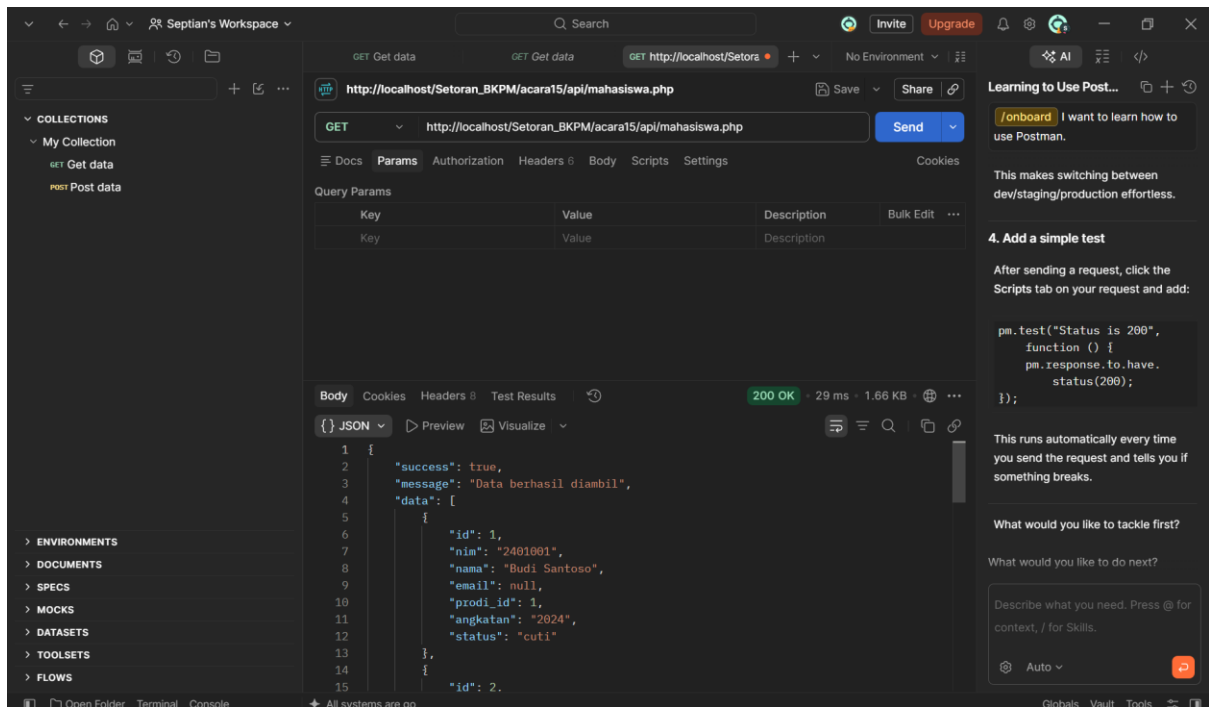
No	Request	Status	Hasil yang diharapkan
1	GET /api/mahasiswa.php	200	data berupa array seluruh mahasiswa
2	GET /api/mahasiswa (tanpa .php)	200	Sama dengan request 1
3	GET ?id=1	200	data berupa object mahasiswa id 1
4	GET ?id=9999	404	Data mahasiswa tidak ditemukan.
5	GET ?id=abc	400	Parameter id harus berupa angka bulat positif.
6	GET ?q=Budi	200	Hanya data yang cocok dengan Budi
7	POST data valid	201	Data tersimpan dan dikembalikan beserta id baru
8	POST NIM 2401001	409	NIM sudah terdaftar.
9	POST data tidak valid	400	errors berisi pesan per field
10	POST body bukan JSON	400	Body harus berupa JSON object yang valid.
11	PUT	405	Method tidak diizinkan

Tabel 4. Skenario pengujian API dan hasil yang diharapkan

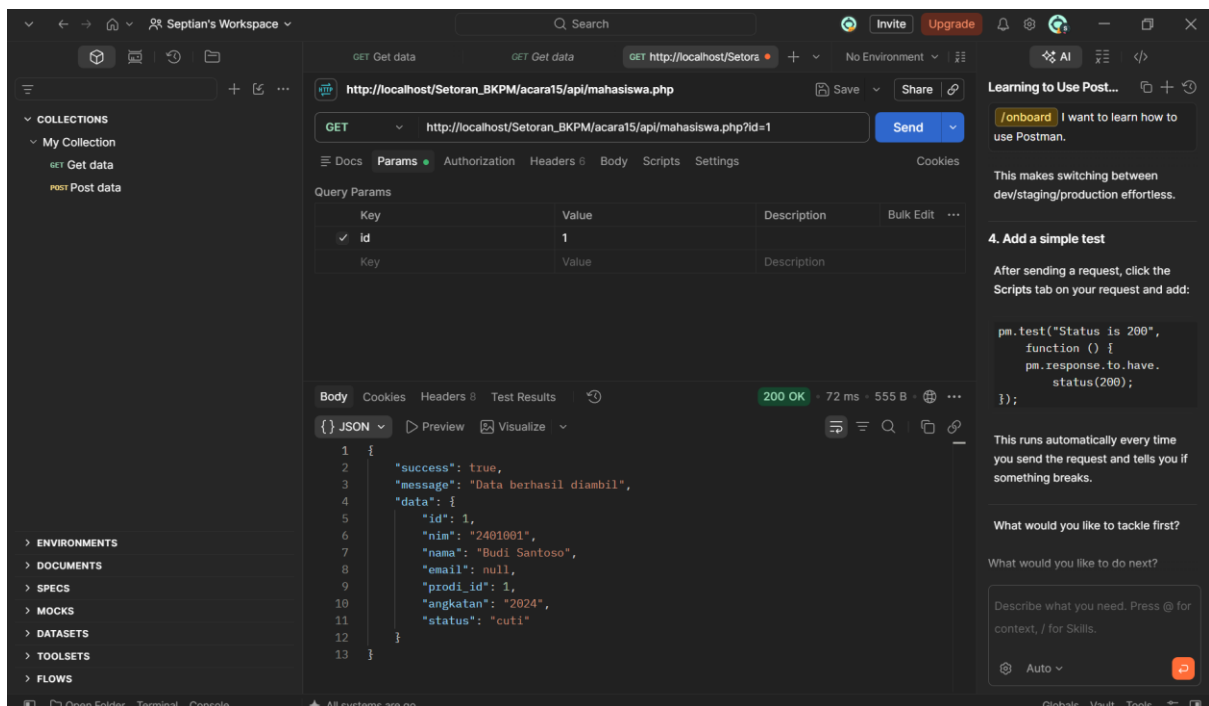


```
{
  "success": false,
  "message": "Terjadi kesalahan pada server."
}
```

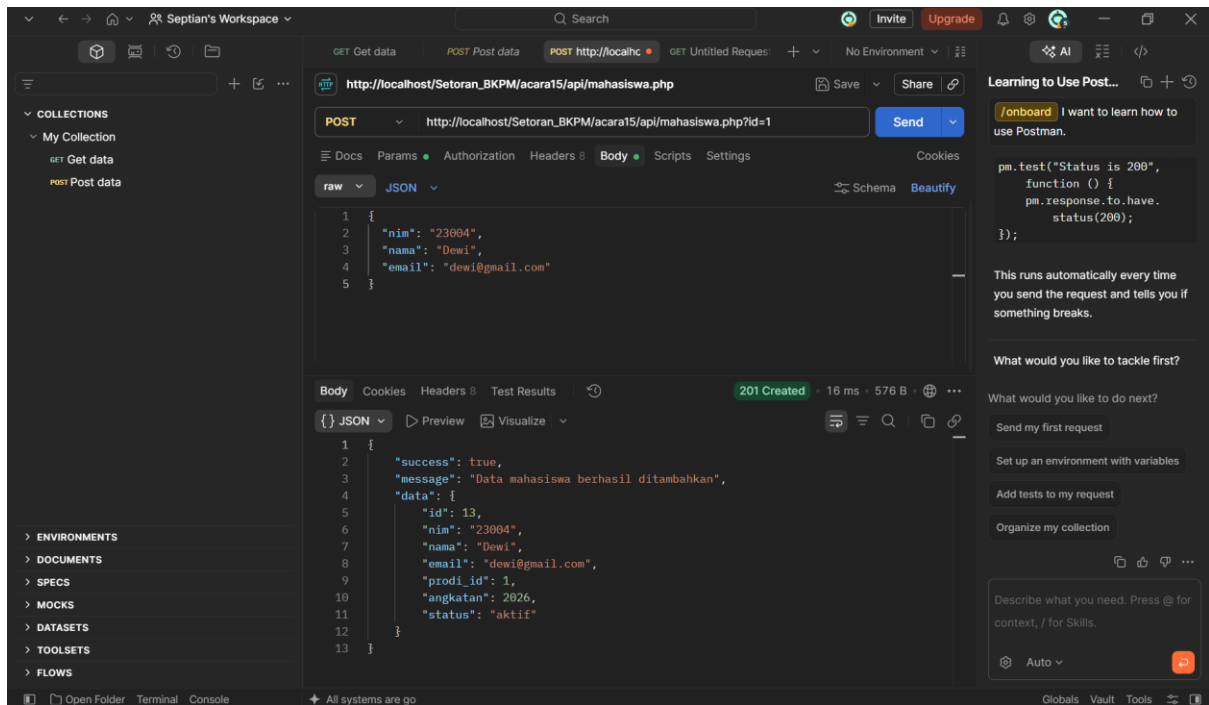
Gambar 19. Pengujian GET melalui browser



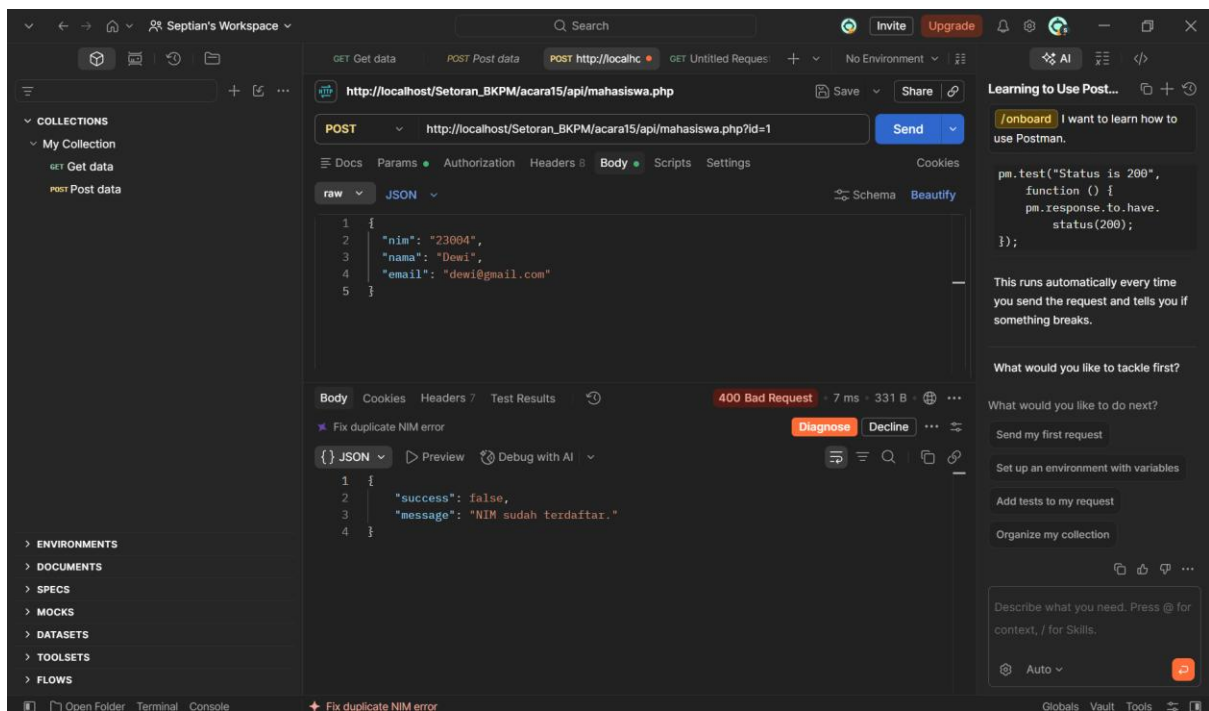
Gambar 20. Pengujian GET daftar mahasiswa dengan Postman



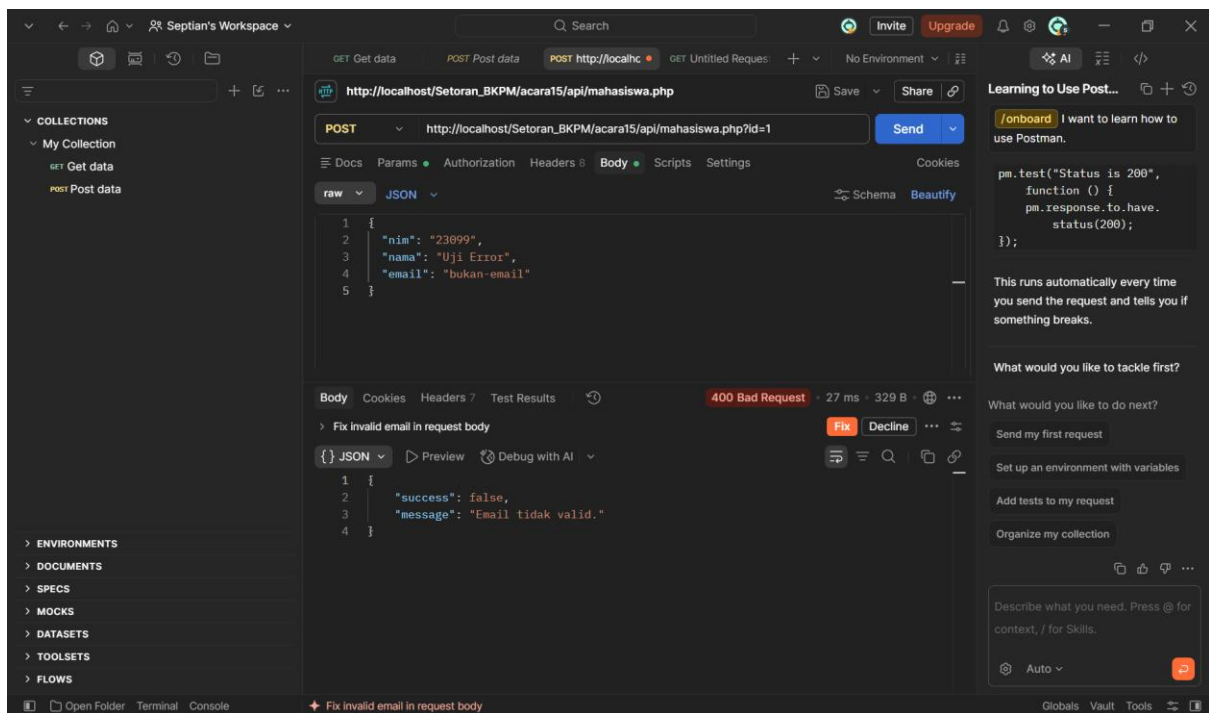
Gambar 21. Pengujian GET satu mahasiswa



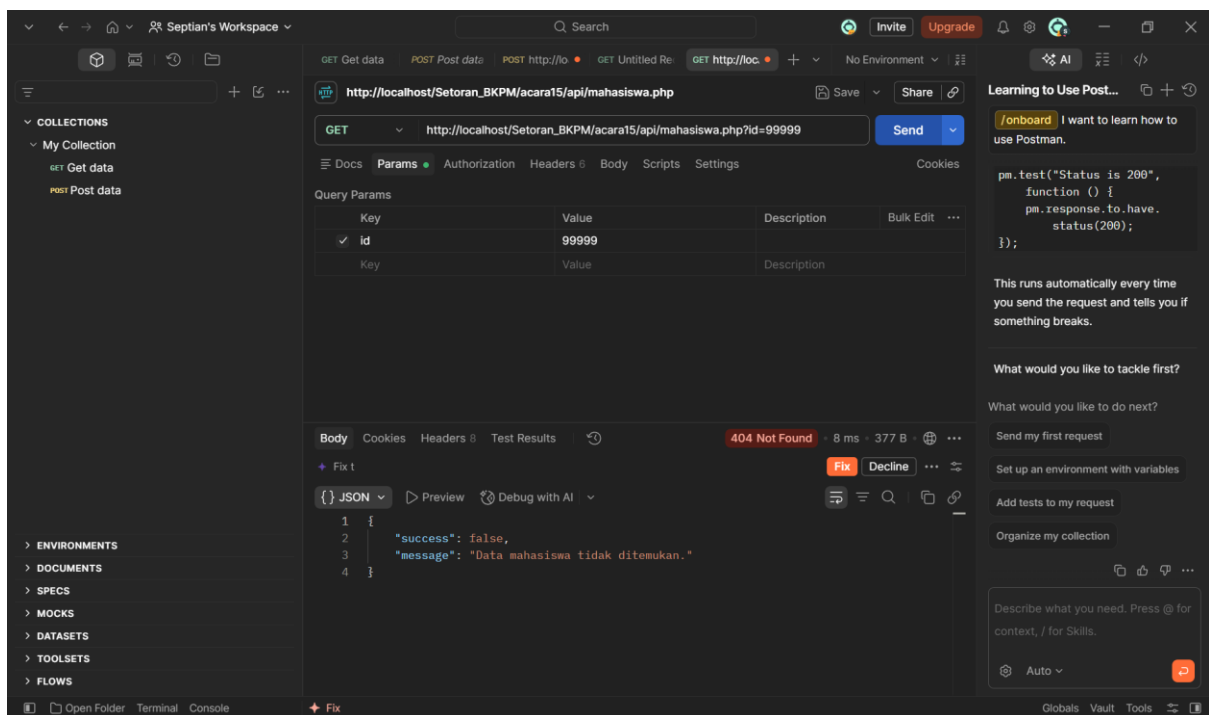
Gambar 22. Pengujian POST tambah mahasiswa



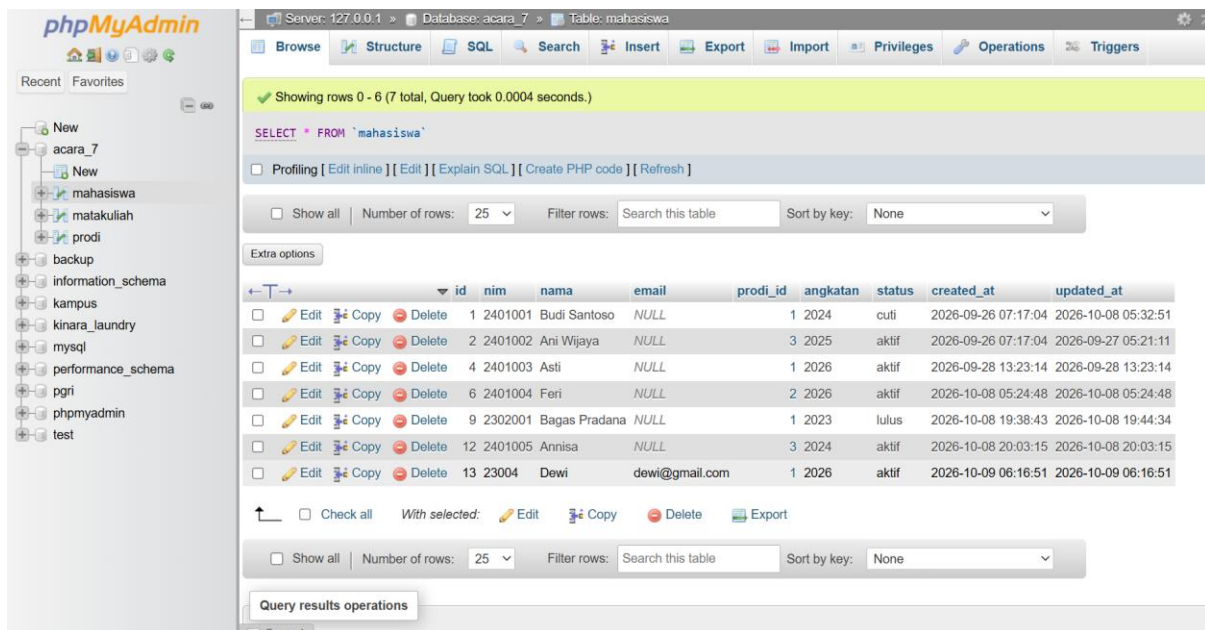
Gambar 23. Pengujian POST NIM duplikat



Gambar 24. Pengujian POST data tidak valid



Gambar 25. Pengujian GET id yang tidak ada



Gambar 26. Bukti data tersimpan di database

6. Keamanan dan Penanganan Error pada API

- Seluruh query memakai prepared statement, termasuk pencarian dan pengambilan berdasarkan id, sehingga parameter di URL tidak dapat mengubah query.
- Parameter id divalidasi (hanya angka bulat positif) dan body POST divalidasi bentuk serta tipenya sebelum diserahkan ke Service.
- Pesan error ke client selalu umum. Detail kesalahan database hanya tercatat di storage/logs/app.log.
- display_errors dimatikan agar warning PHP tidak merusak format JSON maupun membocorkan informasi.
- Method yang tidak didukung dijawab 405 dengan header Allow.

7. Analisis dan Pembahasan

API pada praktikum ini tidak menulis ulang akses database. Endpoint hanya menerjemahkan HTTP request menjadi pemanggilan Service dan menerjemahkan hasilnya menjadi response JSON. Dengan begitu aturan validasi NIM, pengecekan duplikat, prepared statement, dan logging tetap satu sumber kebenaran: perubahan aturan di Service otomatis berlaku untuk aplikasi web maupun API. Hal ini menunjukkan manfaat pemisahan Controller, Service, dan Repository dari Acara 13.

Format response yang seragam (success, message, data, errors) memudahkan client. Client cukup memeriksa field success dan membaca data atau errors, tanpa menebak bentuk response yang berbeda-beda. HTTP status code melengkapi informasi tersebut: 201 untuk data baru, 400

untuk request yang salah, 404 untuk data yang tidak ada, 409 untuk konflik NIM, dan 405 untuk method yang tidak didukung. Pemilihan 409 untuk NIM duplikat dilakukan karena kesalahannya bukan pada bentuk request, melainkan konflik dengan data yang sudah ada.

Endpoint POST tidak menerima begitu saja hasil `json_decode()`. Body yang bukan JSON, bukan object, atau berisi nilai bersarang ditolak dengan 400 sebelum mencapai Service, karena nilai bersarang dapat memicu konversi tipe yang tidak diinginkan. Pada versi ini API belum memiliki autentikasi, sehingga siapa pun dapat melakukan POST. Hal ini dapat diterima untuk tujuan latihan, tetapi pada sistem nyata perlu ditambahkan mekanisme autentikasi seperti token, serta pembatasan akses asal (CORS) bila API dipakai frontend terpisah.

KESIMPULAN

1. API memungkinkan aplikasi lain mengambil data mahasiswa melalui HTTP tanpa mengakses database langsung, sedangkan JSON menjadi format pertukaran datanya.
2. Endpoint `api/mahasiswa.php` berhasil dibuat dan mengambil data dari MySQL (database `acara_7`) melalui PDO, Service, dan Repository dengan prepared statement.
3. Hasil query diubah menjadi JSON dengan `json_encode()` dan header `Content-Type: application/json`, dalam format terstruktur `success`, `message`, dan `data`.
4. Endpoint GET daftar, GET berdasarkan id, dan POST dengan body JSON berhasil dibuat dan diuji menggunakan browser dan Postman, lengkap dengan HTTP status code yang sesuai.
5. Pekerjaan dicatat pada branch `fitur-api-json` yang di-merge ke `main`, dengan koleksi Postman disertakan pada repository.

DAFTAR PUSTAKA

Buku Kerja Praktik Mahasiswa (BKPM) Workshop Sistem Informasi Web Server (TIF330805). Program Studi D4 Teknik Informatika PSDKU Nganjuk, Politeknik Negeri Jember.